

Make idiomatic usage of `offsetof` well-defined

Document #: P3407R1
Date: 2025-01-11
Project: Programming Language C++
Audience: EWG
Reply-to: Brian Bi
[<bbi10@bloomberg.net>](mailto:bbi10@bloomberg.net)

Contents

- 1 Abstract
- 2 Revision history
- 3 Introduction
- 4 Shouldn't this be fixed by the "Accessing object representations" paper?
- 5 Problem: data members are not reachable from other data members, except the first
- 6 Reachability is not about pointer arithmetic
- 7 Provenance in C
- 8 Provenance in C++
- 9 Removing undefined behavior and making optimizations opt-in
- 10 Design space for a solution
 - 10.1 Which pointer types can be used for the pointer arithmetic?
 - 10.2 What about casts to `char*` that are already well-defined?
 - 10.3 Should we just standardize `container_of`?
 - 10.4 Should non-standard-layout types be supported?
- 11 Proposed wording
- 12 Appendix A
- 13 References

§ 1 Abstract

I propose a change to the core language specification that would make it well defined to compute a pointer to the beginning of an object from a pointer to one of its data members (*i.e.* by subtracting the offset of the data member, as given by the `offsetof` macro). Such code, which is often written in C, arguably had well defined behavior prior to C++17. The proposed change will standardize existing practice and is anticipated to have no impact on existing C++ compilers, but will eliminate the possibility of certain (as yet unimplemented) hypothetical reachability-based optimizations that were made possible by the C++17 wording.

§ 2 Revision history

- **R0**, 2024-10-14: Initial revision.
- **R1**, 2025-01-10: Applied wording improvements discovered during CWG review of [P1839R6], including the prohibition on accessing object representations of volatile objects. Improved readability of prose. Added discussion of non-standard-layout types and the possibility of standardizing `container_of`. Added a table comparing the three main design alternatives discussed in this paper.

§ 3 Introduction

In C, an intrusive data structure, such as a doubly-linked list, must be implemented using composition, not inheritance, since C does not have inheritance. Given a pointer to a node within the data structure, accessing the rest of the object requires the use of `offsetof`:

```
struct ListNode {
    struct ListNode* prev;
    struct ListNode* next;
};

typedef struct {
    int data;
    struct ListNode node;
} Foo;

Foo* next_foo(Foo* foo) {
    struct ListNode* next_node = foo->node;
    return (Foo*)((char*)next_node - offsetof(Foo, node));
}
```

This pattern of casting to `char*`, subtracting the appropriate `offsetof` value, and then casting to a pointer to the enclosing type, is often encapsulated in a macro that is named `container_of` or similar (see *e.g.* [GitHub code search](#))¹.

A C++-only project would typically make `ListNode` a base class. Converting a `ListNode*` to a `Foo*` could then be done easily using `static_cast`, and `offsetof` would be unnecessary. This option is not available in C. In C, the `container_of` pattern is the only

option, unless the `ListNode` can be arranged to always be the first member of the enclosing struct.

Unfortunately, the operand of the `return` statement in `next_foo` has undefined behavior in C++. This incompatibility between C and C++ should be fixed, and can be fixed without changing any current C++ compilers.

§ 4 Shouldn't this be fixed by the “Accessing object representations” paper?

At the November, 2019 WG21 meeting, EWG approved [P1839R1] in the following poll:

It should be possible to access the entire object representation through a pointer to a char-like type as a DR

Something like P1839 is certainly needed in order to allow the code given in the previous section to be valid. Currently, casting `next_node` to type `char*` does not yield a pointer that points into an array of `char`; therefore, subtracting any value other than 0 can only have UB (§7.6.6 [expr.add]²p4.3). To solve this problem, [P1839R7] proposes that object representations be made arrays of `unsigned char` (and that pointers to `char` also be allowed to traverse such arrays). This issue has also been pointed out by [P2883R0], which also noted that, although this use of `offsetof` has UB in C++, every known C++ implementation “consistently produced the same behavior as the C program”.

However, EWG did not discuss the issue of *reachability*. Therefore, recent revisions of P1839 have been designed to preserve reachability-based restrictions that currently exist in C++. To put it another way, if P1839 is adopted, it will not change *which* bytes a piece of code is allowed to access, *i.e.*, bytes that it would already be able to access by calling `memcpy`. In order to allow the code given in the Introduction to have well defined behavior, we must expand the set of bytes that are considered reachable from a pointer to a data member.

To be clear, P1839 *could* just make the example in the Introduction valid, but this is a separate evolutionary question from the approved direction in P1839. Therefore, the reachability issue has been made the subject of *this* paper instead of being added to P1839.

§ 5 Problem: data members are not reachable from other data members, except the first

Reachability was introduced into C++17 by the adoption of [P0137R1]. The definition of reachability is currently given in §6.8.4 [basic.compound]p6:

A byte of storage *b* is *reachable through* a pointer value that points to an object *x* if there is an object *y*, pointer-interconvertible with *x*, such that *b* is within the storage occupied by *y*, or the immediately-enclosing array object if *y* is an array element.

The cumulative effect of all changes in P0137R1 was to make it impossible for a pointer derived from a given pointer value, *p*, to access bytes that are not reachable from *p*. The adoption of that paper therefore gave the Committee’s blessing to compiler optimizations based on the *assumption* that unreachable bytes cannot be accessed at all. Unfortunately, in the `next_foo` example given in the Introduction, the bytes constituting the `foo->data` member are not reachable from a pointer to `foo->node`.

For example, assuming `ListNode` and `Foo` have been defined as above, consider the following.

```
void access_node(ListNode* p);

int use_foo() {
    Foo foo;
    foo.data = 1;
    foo.node.prev = &foo.node;
    foo.node.next = &foo.node;
    access_node(&foo.node);
    return foo.data;
}
```

When the body of `use_foo` is compiled, the compiler is allowed to assume that `foo.data` cannot be modified by `access_node`, even though `access_node` is given a pointer to *another* member of the `foo` object. In order to allow `access_node` to access the `data` member through `offsetof` and pointer arithmetic, we must also take away the possibility that a conforming implementation could unconditionally optimize the `return` statement to `return 1;`.

The allowance of this particular reachability-based optimization conflicts with the more important goal of allowing the code given in the Introduction, which would have well defined behavior in C, to have the same behavior in C++. In addition, such optimizations do not seem to have been implemented in any real C++ compilers. Therefore, the changes I propose will not have any impact on existing implementations. I will give more detail in the “Provenance in C++” section below.

§ 6 Reachability is not about pointer arithmetic

In the status quo (prior to the adoption of [P1839R7], if any), reachability can prevent some memory accesses even when no pointer arithmetic is involved. For example:

```
struct S {
    int a[2];
```

```

    int data;
};

void f1(int* p);

int f2() {
    S s;
    s.data = 1;
    f1(&s.a[0]);
    return s.data;
}

```

If `f1` is defined as follows:

```

void f1(int* p) {
    reinterpret_cast<S*>(reinterpret_cast<int (*)[2]>(p))->data = 2;
}

```

then calling `f1` has undefined behavior, because the entire array `s.a` is not pointer-interconvertible with the element `s.a[0]`. The inner `reinterpret_cast` yields a “wrongly typed” pointer: a pointer value that is of type `int (*)[2]`, but points to a single `int`, namely `s.a[0]`; it does not point to the array `s.a`. Consequently, the outer `reinterpret_cast`, which attempts to go from the first member of a standard-layout struct to the struct itself (allowed in C++17), cannot work; instead another wrongly typed pointer is produced: a value of type `S*` that points to `s.a[0]` (not `s`). Dereferencing this pointer yields an lvalue that does not refer to an `S` object, which renders the attempted access to `data` UB (§7.6.1.5 [expr.ref]p9).

The `std::launder` function, which can accept a pointer and return a different pointer value that holds the same address, does not help, because it has a reachability restriction: calling `std::launder` on a wrongly typed pointer picks out the object of the correct type that lives at the address that the pointer holds, but if there are bytes reachable from that object that are not reachable from the object that the original pointer points to, the behavior is undefined (§17.6.5 [ptr.launder]p2).

Therefore, the implementation can assume that the call to `f1` in `f2` never modifies `s.data`: if any attempt were made to do so, then the behavior of the program would be undefined.

In [P1839R7], I have attempted to ensure that the proposed wording is consistent with the reachability restrictions that exist in current C++, because there is no record of EWG having discussed the question of whether those restrictions should be relaxed. If the `get_next_foo` example is to be made well-defined, then some reachability-based assumptions that are currently allowed to implementations must be invalidated. This paper proposes to do just that.

§ 7 Provenance in C

The C standard does not currently have a notion of *provenance*, but it is widely assumed that one ought to exist. For example, in the following translation unit:

```

void evil(void);
int main(void) {
    int x = 1;
    evil();
    return x;
}

```

notwithstanding that `evil` might be able to “guess” the address of `x` based on knowledge of the platform ABI, it is widely agreed that `evil` should be allowed to neither read nor write the value of `x`, and, therefore, the compiler can eliminate `x` and optimize the last statement to `return 1;`. GCC and Clang both perform this optimization at `-O1` and higher.

One can say that even if `evil` correctly guesses the numerical value of `x`’s address, casting that numerical value to `int*` would yield a pointer that *lacks provenance* and, therefore, causes UB when dereferenced. Such provenance-based restrictions on the use of pointers do not exist in the current C standard, but work is underway on a Draft Technical Specification for pointer provenance in C (referred to as the “Provenance TS” from this point onward). The latest version of the Provenance TS is [N3057].

In the Provenance TS, values of pointer-to-object type³ are augmented to include provenance, which may be empty. A non-empty provenance is the ID of a *storage instance*, and a pointer value whose provenance is the ID of a storage instance *I* can be used only to access bytes that lie within *I*. In the example above, a storage instance is created when `x` is defined. In contrast to the address that a pointer value represents, there is no way to directly change the provenance of a pointer, other than by storing into it another pointer value that has the desired provenance. That is, no cast or other operation in `evil` can construct a pointer value whose provenance is the ID of `x`. Therefore, the implementation can assume that any pointer constructed by `evil` that happens to represent the address of `x` cannot be used to access `x`, since the provenance of such a pointer value is either empty or a storage ID other than that of `x`.

Although the Provenance TS doesn’t explicitly state that subobjects have the provenance of their complete object, the definition of “storage instance” given in section 3.20 of Annex C implies that only a single storage instance is created by an object definition. A note to section 3.20 states that two subobjects within an object of structure type share a storage instance.

Therefore, under the Provenance TS, if the address of a subobject is taken, the resulting pointer’s provenance is a storage ID that contains at least the complete object⁴. Therefore, all bytes of a complete object are always reachable starting from a valid pointer to any subobject.

§ 8 Provenance in C++

C++ has had a provenance-based pointer model since [P0137R1]. However, the C++ standard does not use the term “provenance”. Instead, every dereferencable pointer in C++ has a unique object or function to which it points. But the set of bytes that an object

pointer can reach is not necessarily limited to the bytes occupied by the object that the pointer points to. For example, a pointer to any element of an array can be used to access any byte of the array, including bytes that are occupied by other elements. C++ is more restrictive than the C Provenance TS: all bytes reachable from the pointer value “pointer to *o*” (where *o* is an object) lie within *o*’s complete object, but not all bytes of a complete object are reachable from a pointer to a subobject. In particular, as stated previously, if a pointer points to a non-static data member of a standard-layout struct other than the first non-static data member, no other members are reachable from that pointer.

To look at it from the point of view of the compiler, all provenance-based optimizations that are valid in C are also valid in C++. For example, Clang, GCC, and MSVC are all capable of performing the optimization mentioned in the previous section (*i.e.* that the value of *x* is not accessed by *evil*). Since C++ is stricter than C, some provenance-based optimizations that are not valid in C are valid in C++. However, **I have not been able to find any cases in which C++ implementations exploit provenance-based optimizations that are not valid in C.** For example, in the following translation unit:

```
struct S {
    int x;
    int y;
};
void f4(int* p);
int f3() {
    S s;
    s.x = 1;
    f4(&s.y);
    return s.x * s.x;
}
```

even at maximum optimization levels, Clang, GCC, and MSVC all generate a load of *s.x* and an *imul* instruction on x86-64; no implementation assumes that, because only the address of *s.y* escapes from *f3*, the value of *s.x* cannot be changed.

I believe that the reason why such optimizations are not performed is that C++ implementations wish to maintain a reasonable degree of compatibility with C. Since C code often uses the *container_of* idiom, which could be used to obtain a pointer to *s* given a pointer to *s.y*, implementations make allowances for the same operation to take place in a C++ program. Therefore, not only do implementations not currently perform this optimization, but it is unlikely that future versions will do so, either. Implementations are more constrained by the needs of their users, in this case, than by the availability of compiler engineers to implement the optimization.

Similarly, the function *f1* defined earlier could be given the following definition in C. The offset value will always be 0 in this case, so the subtraction can be omitted without changing the meaning.

```
void f1(int* p) {
    (S*)((char*)p - offsetof(struct S, a[0]))->data = 2;
}
```

Therefore, even in C++ mode, implementations do not assume that `f1` cannot change the value of `data`, even though the reachability rules of the language permit optimizations based on this assumption. Clang, GCC, and MSVC all emit both a store to `s.data` before the call to `f1` and a load after.

§ 9 Removing undefined behavior and making optimizations opt-in

The overly strict reachability rules adopted in C++17 have an additional disadvantage besides limiting compatibility with C: they create a category of constructs that:

1. A programmer can easily use without realizing that UB will result, and
2. Can be given perfectly sensible defined behavior (which may include implementation-defined or unspecified results) only at the cost of minor optimizations.

My opinion is that the Committee should not create new forms of UB that meet the above criteria, and should strongly consider removing any such UB that already exists in the language. UB that is actually exploited by compilers for optimization purposes makes the use of C++ less safe; UB that is not currently exploited still has a negative impact on the perception of how safe C++ is, and is scary to beginners, who don't have enough context to distinguish between benign UB that is unlikely to ever be exploited and dangerous UB that may eventually result in an unbounded set of possible executions.⁵ I do not mean to suggest that all or even most UB can be removed from C++, but when the two criteria above are met, I think the cost/benefit analysis heavily favors giving the construct a defined behavior.

I believe that a better way to obtain the optimizations that such UB is meant to enable is to provide mechanisms to opt in: that is, language or library features whose sole purpose is to cause UB, which can then be used to optimize; experts can use such features to produce faster code, while beginners can easily avoid them because they cannot be used by accident while writing code that uses other C++ features. (The `[[assume]]` attribute is a well-known example of this genre.) It seems much more defensible to provide “sharp tools” for experts to use in order to improve performance than to build sharp edges into the most basic language constructs, making it difficult for beginners to use them safely.

Consider again this example from the previous section:

```
struct S {  
    int x;  
    int y;  
};  
void f4(int* p);  
int f3() {  
    S s;  
    s.x = 1;  
}
```



```
f4(&s.y);  
return s.x * s.x;  
}
```

This paper proposes that `f4` would have the ability to modify `s.x`, and that if there is sufficient interest from C++ experts in having a way to tell the compiler that `s.x` *cannot* be reached through the pointer passed to `f4`, a new mechanism can be added to the language. This possibility is discussed in Appendix A.

§ 10 Design space for a solution

To make the C++ standard match existing practice of implementations and to bless `container_of`-like constructs in C++, it is necessary to permit pointer arithmetic within objects, which is already being proposed by [P1839R7], and also to relax the reachability rules in C++. However, this paper does not propose to relax the C++ reachability rules all the way to the “complete object or allocation” model proposed by the C Provenance TS because doing so is not necessary to solve the immediate problem. Instead, it suffices to allow a pointer to an object to reach all bytes of the complete object. For example, this paper does not propose to enable the use of flexible array members in C++, which are allowed by the C Provenance TS because the trailing bytes belong to the same storage instance (allocation) as the preceding members. The `container_of` technique was valid in C++ prior to C++17 and this paper aims to restore the *status quo ante*, not to propose a new feature that has never been in C++.

§ 10.1 Which pointer types can be used for the pointer arithmetic?

Because typical `container_of` macros in C use a cast to `char*` (not `unsigned char*`), this paper proposes that a cast to `char*` be allowed to yield a pointer to an object’s object representation; pointers to `unsigned char` and `std::byte` are also supported, as these types are already exempt from the strict aliasing rule (§7.2.1 [basic.lval]p11).

§ 10.2 What about casts to `char*` that are already well-defined?

In some cases, a C-style cast to `char*` already has well-defined behavior in C++ that is different than producing a pointer to the object representation. One of these cases is when the operand points to an object of class type that has a conversion function to cv `char*`. I do not propose to change the behavior of such casts in C++; doing so would be a disastrous breaking change that is not needed for C compatibility, because C does not have conversion functions. The remaining two cases are:

1. The cast is a `const_cast` because the operand has type cv `char*` or array of cv `char`. (This includes the case where no conversion is needed at all.)

2. The cast can be interpreted as a `reinterpret_cast` followed by an optional `const_cast` because there is a “real” cv `char` (not an element of an object representation) that is located at the address represented by the operand and is pointer-interconvertible with it.

I searched GitHub for uses of `container_of` and uses of `offsetof` for the purpose of reaching an enclosing struct. In the 65 files that I analyzed manually, I found two files in which the pointer from which the `offsetof` value is subtracted points to an array of `char`. (In one of these cases, the array was a flexible array member, which is not part of standard C++, but is often accepted as an extension.) That is, the relevant details of the code are similar to:

```
struct S2 {
    int data;
    char buf[100];
};
int get_data(char* p) {
    return ((struct S2*)(p - offsetof(S2, buf)))->data;
}
void f5() {
    S2 s;
    // ...
    get_data(s->buf);
    // ...
}
```

In C++, this code performs out-of-bounds array arithmetic, and thus exhibits UB even before the attempt to access data.

Essentially, this gives us three design options to deal with Case 1.

1. We can say that cv `char*` is exempt from bounds checking, just as it's exempt from the strict aliasing rule. In other words, while a `char*` may point to a specific `char` object during constant evaluation, in all other cases it merely points to a byte of storage, and pointer arithmetic that would reach any other byte in the same complete object is permitted. In this case, cv `unsigned char*` would also be exempt from bounds checking (for symmetry with the strict aliasing rule). This might have a negative impact on performance relative to the status quo if compilers are currently relying on the assumption that a pointer into a `char` array that is a subobject cannot be used to perform pointer arithmetic outside the bounds of the array. However, I have not yet found any examples where compilers do use such assumptions for optimization. The more likely impact is on sanitizers and static analyzers: they might be forced to disable bounds checking for `char*` and `unsigned char*`, which would reduce their ability to detect UB.
2. We can say that a C-style cast from `char*` to `char*` or a similar cast (as described in Case 1) sometimes *changes* the pointer value such that the above example would have defined behavior if `p` were to be cast to `char*` prior to the pointer arithmetic. (A similar allowance would be made for casts to `unsigned char*`.) In many cases

in the real world, such a cast might be present because it will have been introduced by a generic `container_of`-like macro that is not “aware” of the fact that the pointer argument, in some particular cases, has type `char*` already. However, this design has two disadvantages. First, some compilers might simply ignore casts from `char*` to `char*` at some early stage of semantic analysis, so that at some later stage they are not aware that the cast is there at all, so the cast cannot achieve its purpose of giving the program defined behavior; it is not clear how much work would be required to change the implementations. Second, it would violate the current design in which a C-style cast is equivalent to trying C++-style casts in a particular order; instead, the C-style cast would have the additional power of producing a pointer to the object representation instead of performing a no-op `const_cast`.

3. We can say that we don’t care enough about solving the problem in the case of pointers that are already of type `char*`. The example above would continue to have UB, regardless of whether an additional cast is inserted. We would still solve 99% of the problem, because in the vast majority of cases, the subobject pointer points to an object of struct or union type, not a `char`.

To understand the practical implications of these options, consider the following three possible definitions of `get_data`:

- A. `return ((struct S2*)(p - offsetof(S2, buf)))->data;`
- B. `return ((struct S2*)((char*)p - offsetof(S2, buf)))->data;`
- C. `return ((struct S2*)((unsigned char*)p - offsetof(S2, buf)))->data;`

In the status quo, even assuming [P1839R7] is adopted, the behavior is undefined in C++ in all three cases (A, B, and C). Some or all of these cases are well defined under C⁶ and under the proposed options discussed above:

Case	ISO C with Provenance TS	Option 1	Option 2	Option 3
A	Undefined	Well defined	Undefined	Undefined
B	Undefined	Well defined	Well defined	Undefined
C	Well defined	Well defined	Well defined	Well defined

This paper proposes option 3 as the conservative option, without prejudice to adopting something similar to option 1 or 2 in the future. The status of Case A and Case B in ISO C is a bit unclear; if the C standard were to be clarified to make Case A and Case B definitely well defined, their specification strategy could provide inspiration for a corresponding change in C++ to preserve compatibility. Assuming such a change is not made, the small amount of C code that is similar to Case A or Case B could be rewritten so that, if the subobject has type `char`, then the pointer arithmetic is done using `unsigned`

`char*` (Case C), and *vice versa*; it would then have the desired behavior in both C and C++ under option 3.

For Case 2, I also found two examples in the 65 files that I analyzed in which the subobject pointer points to a struct that is pointer-interconvertible with an `unsigned char` subobject. I didn't find any examples with `char`, but given that examples exist that use `unsigned char`, I assume there are others that use `char`. Such code would have relevant details similar to:

```
struct S3 {
    char a;
    int b;
};
struct S4 {
    char c;
    struct S3 d;
};

struct S4* get_s4(struct S3* s3) {
    return (struct S4*)((char*)s3 - offsetof(S4, d));
}
```

In current C++, the cast to `char*` yields a pointer to the a subobject. Note, however, that the entire example has undefined behavior because of the subsequent pointer arithmetic. If we change the rules of C++ so that the cast would be allowed to yield a pointer to the object representation of the S4 object, we could make this example well-defined when it currently is not. In order to avoid changing the behavior of any code that is already well-defined, we could say that the status quo interpretation of the cast takes precedence, and an pointer to the object representation is obtained only when the former interpretation would produce undefined behavior. This specification strategy is similar to that of implicit object creation, in which the specific objects that are created may only be determined by the details of a later operation, which would have UB other than under one particular choice of objects to create.

§ 10.3 Should we just standardize `container_of`?

Standardizing `container_of` could provide a solution to the `char*/unsigned char*` problem: if the operand is of type `char*`, it would be cast to `unsigned char*`, and vice versa, thus ensuring that a pointer to the object representation can always be obtained (In C, this behavior can be implemented using a `_Generic` expression.) Cases A and B in the previous subsection would remain undefined but such undefined behavior would not be invoked as long as `container_of` is used. However, pursuing standardization of `container_of` must begin in WG14, not WG21, and in any case, such work would have to be *in addition* to this paper, not instead of it. Casting to the other pointer type is of no use if reachability restrictions are not also relaxed in C++.

§ 10.4 Should non-standard-layout types be supported?

The `offsetof` macro is conditionally-supported when its type argument is not a standard-layout class (§17.2.4 [\[support.types.layout\]](#)p1). Any struct that is valid in C will produce a standard-layout class when its definition is compiled as C++ code. Therefore, for purposes of C++ compatibility, we do not necessarily need to allow all bytes of a non-standard-layout class to be reachable from a pointer to one of its subobjects.

However, limiting the changes in this paper to standard-layout classes has some downsides, and no known upsides:

1. It would complicate the specification.
2. It would leave a footgun in the language. For example, if a codebase originally written in C were converted to be C++-only, and a non-standard-layout member were then added to a standard-layout class, existing code that uses `offsetof` to reach the beginning of the class would silently acquire undefined behavior.
3. Although a derived class object can always be reached (even if the derived class is not standard-layout) from a pointer to a base class subobject, there are situations in which it is desirable for the complete object to have data members instead of base classes, including data members of non-standard-layout types (which cause the enclosing class to be non-standard-layout, too). These reasons, in the context of the C++ execution library, are discussed by Lewis Baker in [\[P3425R0\]](#). Allowing the equivalent of `container_of` to be used in C++ would simplify the use case discussed therein. (And perhaps the C++ standard should be amended to require `offsetof` to be supported for non-standard-layout types that are aggregates, but that's beyond the scope of this paper.)

§ 11 Proposed wording

This wording is a modified version of the wording in [\[P1839R7\]](#) and is relative to working draft [\[N5001\]](#).

Modify §6.7.2 [\[intro.object\]](#)p3 as follows:

If a complete object is created (`[expr.new]`) in storage associated with another object *e* of type “array of *N* `unsigned char`” other than a synthesized object representation ([\[basic.types.general\]](#)) or of type “array of *N* `std::byte`” ([\[cstddef.syn\]](#)), that array *provides storage* for the created object if [...]

Modify §6.7.2 [\[intro.object\]](#)p4 as follows:

An object *a* is *nested within* another object *b* if

- *a* is a subobject of *b*, or
- *b* provides storage for *a*, or
- *a* and *b* are the object representations of two objects *o1* and *o2*, where *o2* provides storage for *o1*, or

- there exists an object *c* where *a* is nested within *c*, and *c* is nested within *b*.

Modify §6.7.2 [intro.object]p10 as follows:

Unless an object is a bit-field or a subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects with overlapping lifetimes that are not bit-fields may have the same address if

- one is nested within the other,
- at least one is a subobject of zero size and they are not of similar types ([conv.qual]),~~or~~
- at least one is a synthesized object representation or element thereof, or
- they are both potentially non-unique objects;

otherwise, they have distinct addresses and occupy distinct bytes of storage.

Modify §6.7.2 [intro.object]p14 as follows:

Except during constant evaluation, an operation that begins the lifetime of an array of `unsigned char` or `std::byte` other than a synthesized object representation ([basic.types.general]) implicitly creates objects within the region of storage occupied by the array.

Edit §6.7.4 [basic.life]p1 as follows:

[...] The lifetime of an object of type *T* other than an element of a synthesized object representation ([basic.types.general]) begins when

- storage with the proper alignment and size for type *T* is obtained, and
- if it is not a synthesized object representation, its initialization (if any) is complete (including vacuous initialization) ([dcl.init]),

except [...]. The lifetime of an object *o* of type *T* other than an element of a synthesized object representation ends when:

- if *T* is a non-class type, the object is destroyed, or
- if *T* is a class type, the destructor call starts, or
- the storage which the object occupies is released, or is reused by an object that is ~~not~~neither nested within *o* ([intro.object]) nor nested within the object of which *o* is the object representation, if any ([basic.types.general]).

When evaluating a *new-expression*, [...]

[Example 1: [...] end example]

A synthesized object representation is not considered to reuse the storage of any other object.

Insert a new paragraph after §6.7.4 [basic.life]p3 as follows:

The lifetime of a reference begins when its initialization is complete. The lifetime of a reference ends as if it were a scalar object requiring storage.

[Note 1: [class.base.init] describes the lifetime of base and member subobjects. —end note]

[For an object *o* of class type, the lifetimes of the elements of the synthesized object representation begin when the construction of *o* begins and end when the destruction of *o* completes. Otherwise, the lifetimes of the elements of the synthesized object representation (if any) are the lifetime of *o*.

Modify §6.8.1 [basic.types.general]p4 as follows and add a paragraph after it:

The *object representation* of a complete object type *T* is the sequence of *N* ~~unsigned char objects~~bytes taken up by a ~~non-bit-field~~ complete object of type *T*, where *N* equals `sizeof(T)`. The *value representation* of a type *T* is the set of bits in the object representation of *T* that participate in representing a value of type *T*. The object and value representation of a ~~non-bit-field~~ complete object of type *cv T* are the bytes and bits, respectively, of the object corresponding to the object and value representation of its type; the object representation is considered to be an array of *N cv unsigned char* if the object occupies contiguous bytes of storage ([intro.object]). The object representation of a bit-field object is the sequence of *N* bits taken up by the object, where *N* is the width of the bit-field ([class.bit]). The value representation of a bit-field object is the set of bits in the object representation that participate in representing its value. Bits in the object representation of a type or object that are not part of the value representation are padding bits. For trivially copyable types, the value representation is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values.

For a complete object *o* with type *cv T* whose object representation is an array *A*:

- If *o* has type “array of *cv unsigned char*”, then *A* is *o*.
- Otherwise, *A* is said to be a *synthesized object representation*, and is distinct from any object that is not an object representation.

[Note: In particular, when an array *B* of *N unsigned char* provides storage for an object *o* of size *N*, the object representation of *o* is a different array that occupies the same storage as *B*. —end note]

For each element *e* of *A*:

- If e occupies the same storage as an object having type `cv char`, `cv unsigned char`, or `cv std::byte` that is either o or a non-bit-field subobject thereof, the value of e is the value congruent ([basic.fundamental]) to that of the subobject.
- Otherwise, for each bit b in the byte of o that corresponds to e , let b' be the corresponding bit of e and let $p(b)$ be the smallest subobject of o that contains b , other than an inactive union member or subobject thereof. If $p(b)$ is a union object or is not within its lifetime or has an indeterminate value, or if b is not part of the value representation of $p(b)$, then b' has indeterminate value. Otherwise, if b has an erroneous value, then b' has an erroneous value. Otherwise, b' has an unspecified value that is neither indeterminate nor erroneous; such a bit retains its value until $p(b)$ is subsequently modified.

[Note: Attempting to access an element of a synthesized object representation of a volatile object results in undefined behavior ([dcl.type.cv]). —end note]

[Note: An object representation is always a complete object. —end note]

Modify §6.8.4 [basic.compound]p5 as follows:

Two objects a and b are *pointer-interconvertible* if they have the same address and:

- ~~they are the same object, or~~
- ~~one is a union object and the other is a non-static data member of that object ([class.union]), or~~
- ~~one is a standard-layout class object and the other is the first non-static data member of that object or any base class subobject of that object ([class.mem]), or~~
- ~~there exists an object c such that a and c are pointer-interconvertible, and c and b are pointer-interconvertible.~~
- they have the same complete object, or
- the complete object of one is the object representation of the complete object of the other.

~~If two objects are pointer-interconvertible, then they have the same address, and it is possible to obtain a pointer to one from a pointer to the other via a `reinterpret_cast` ([expr.reinterpret.cast]):~~

[Note: A `reinterpret_cast` ([expr.reinterpret.cast]) never converts a pointer to a to a pointer to b unless a and b are pointer-interconvertible. —end note]

[Note: A standard-layout class object is pointer-interconvertible with its first non-static data member (if any) and each of its base class subobjects ([class.mem]). An array object and an object that the array provides storage for are not pointer-interconvertible. —end note]

Modify §6.8.4 [basic.compound]p6 as follows:

A byte of storage *b* is *reachable through* a pointer value that points to an object *x* if ~~there is an object *y*, pointer-interconvertible with *x*, such that *b* is within the storage occupied by *y*, or the immediately enclosing array object if *y* is an array element~~*b* is within the storage occupied by *x*'s complete object.

Modify §7.2.1 [basic.lval]p11 as follows:

An object of dynamic type T_{obj} is *type-accessible* through a glvalue of type T_{ref} if T_{ref} is similar ([conv.qual]) to:

- T_{obj} ,
- a type that is the signed or unsigned type corresponding to T_{obj} , or
- a ~~char, unsigned char,~~ or `std::byte` type ,if the object is an element of an object representation ([basic.life.general]).

If a program attempts to access ([defns.access]) the stored value of an object through a glvalue through which it is not type-accessible, the behavior is undefined. [...]

[Note 11: [...]]

[Example 2: An element of an object representation can be accessed through a glvalue of type `char`, `unsigned char`, `signed char`, `std::byte`, or a cv-qualified version of any of these types. —end example]

Drafting note: Because we don't guarantee that all complete objects are contiguous (see [P1945R0]) it cannot always be guaranteed that, e.g., a `reinterpret_cast` to `unsigned char*` will yield a pointer to an element of an object representation: no synthesized object representation is present at all in the discontinuous case. In those cases, we do not attempt to specify the behavior of access the original object through a glvalue of char-like type, so we shouldn't claim that it's well defined to do so.

Modify §7.3.2 [conv.lval]p3.4, as amended by the proposed resolution of [CWG2901], as follows:

- Otherwise, the object indicated by the glvalue is read ([defns.access]). Let *V* be the value contained in the object. If *T* is an integer type or cv `std::byte`, the prvalue result is the value of *T* congruent ([basic.fundamental]) to *V*, and *V* otherwise. [...]

Modify §7.6.1.9 [expr.static.cast]p13 as follows:

[...] Otherwise, if the original pointer value points to an object *a*, ~~and there is an object *b* of type similar to *T* that is pointer-interconvertible ([basic.compound]) with *a*, the result is a pointer to *b*. Otherwise, the pointer value is unchanged by the conversion.~~ let *S* be the set of objects that are pointer-interconvertible with *a* and have type similar to *T*.

- If *S* contains *a*, the result is a pointer to *a*.
- Otherwise, the result is a member of *S* whose complete object is not a synthesized object representation if any such result would give the program defined behavior. If there are multiple possible results that would give the program defined behavior, the result is an unspecified choice among them.
- Otherwise (i.e. when there are no such members of *S* that would give the program defined behavior), if the object representation of *a*'s object is an array *A*, *T* is similar to the type of *A*, and *A* is a member of *S*, the result is a pointer to *A*.
- Otherwise, if the object representation of *a*'s complete object is an array and *T* is `cv unsigned char`, the result is a pointer to the element of that object representation that has the same address as *a*.
- Otherwise, if *T* is `cv char` or `cv std::byte`, or an array of one of these types, let *U* be the type obtained from *T* by replacing `char` or `std::byte` with `unsigned char`. If a `static_cast` of the operand to *U** would be well-formed and would yield a pointer to an object representation or element thereof, the result of the cast to *T** is that pointer value.
- Otherwise, the result is a pointer to *a*.

Otherwise, if the original pointer value points past the end of an object *a*:

- If the object representation of the complete object of *a* is an array *A*, *T* is similar to the type of *A*, and *a* has the same address as *A*, the result is `&A+1`.
- Otherwise, if the object representation of the complete object of *a* is an array *A* and *T* is `cv unsigned char`, the result is a pointer to the element of *A* (possibly the past-the-end element) that has the same address as the one represented by the operand.
- Otherwise, if *T* is `cv char` or `cv std::byte`, or an array of one of these types, let *U* be the type obtained from *T* by replacing `char` or `std::byte` with `unsigned char`. If a `static_cast` of the operand to *U** would be well-formed and would yield a pointer value defined by one of the above cases, the result of the cast to *T** is that pointer value.
- Otherwise, the result is the value of the operand.

Modify §7.6.6 [\[expr.add\]](#)p6 as follows:

For addition or subtraction, if the expressions P or Q have type “pointer to cv T”, ~~where T and the array element type are not similar, the behavior is undefined.~~ , one of the following shall hold:

- T is similar to the array element type, or
- T is similar to char or std::byte and the pointer value points to a (possibly-hypothetical) element of an object representation.

Otherwise, the behavior is undefined.

Modify §9.2.9.2 [dcl.type.cv]p5 as follows:

If an attempt is made to access an element *e* of a synthesized object ([basic.types.general]) and *e* overlaps the storage occupied by a volatile object (including a subobject) that is within its lifetime, the behavior is undefined. Otherwise, the ~~The~~ semantics of an access through a volatile glvalue are implementation-defined. If an attempt is made to access an object defined with a volatile-qualified type through the use of a non-volatile glvalue, the behavior is undefined.

§ 12 Appendix A

The C programming language already contains an opt-in feature that can be used to tell the compiler that a pointer to part of an object cannot be used to access other parts of the same object. That feature is the restrict keyword. Using restrict, the definition of the function f3 given previously could be changed to:

```
int f3(void) {
    struct S s;
    s.x = 1;
    {
        struct S* restrict p = &s;
        f4(&s.y);
        return p->x * p->x;
    }
}
```

In the example above, if `s.x` is accessed through an lvalue that is *based on* the restricted pointer `p` **and** `s.x` is modified at any point during the execution of the block in which `p` is defined, then all accesses to `s.x` during that block must be through lvalues that are based on `p`. The first condition (that `s.x` is accessed through an lvalue based on `p`) is already met by the return statement in `f3`; the second condition will be met if `f4` attempts to modify `s.x`. In that case, all accesses to `s.x` during the lifetime of `p` would need to be through lvalues based on `p`, but the modification in `f4` could not be, so the behavior would be undefined. The compiler can assume that this scenario does not occur, and that `s.x` will still have the value 1 upon return from `f4`.

GCC does not actually perform this optimization, even with `-O3`. I can only speculate as to the reason: I suspect that this is not the kind of optimization that `restrict` was designed to enable, and that such an optimization is simply not very useful. However, let's assume for the sake of argument that some experts would benefit from being given a tool to enable such an optimization in C++: one that (unlike the current reachability rules in C++) could actually be used by implementations without breaking compatibility with C. What might that tool look like? `restrict` itself is unlikely to be added to C++. If we were to design a different feature for this purpose, we would probably want it to be in a form that could also be added to C.

For example, we could change the definition of pointer values in the C++ standard so that, in the case of an object pointer, the value not only identifies the object that the pointer value points to or past the end of, but also includes a *reachable range*, which is a contiguous set of bytes; a pointer could be used to access memory only at addresses that lie within the pointer value's reachable range. This provenance model is the one used by CHERI, which refers to the reachable range as the *bounds* of a pointer value. The *CHERI C/C++ Programming Guide* [CHERI] states that the *subobject bounds* feature (described in Section 4.3.3), in which taking the address of a subobject produces a pointer value whose bounds are narrowed to the memory occupied by the subobject, is not enabled by default, and when enabled, breaks code that uses the “container of pattern” (p. 16); such code must be modified to *opt out* of subobject bounds. However, CHERI aims to provide improved safety (e.g., by “[preventing] an overflow on [an array subobject] from affecting the remainder of the structure”); when the objective of narrowing bounds is to create potential UB and enable additional optimizations, an opt-in mechanism is more appropriate. Such an opt-in mechanism, that would be based on Core wording that defines reachable ranges, might be a library function like the following:

```
/// If `p1` is a null pointer, return `p1`. Otherwise, return a pointer
/// that
/// points to or past the end of the same object `o` as `p1` but whose
/// reachable range consists of the bytes in [p2, p3). The storage
/// occupied by
/// `o` shall be a subrange of [p2, p3), which shall be a subrange of the
/// reachable range of `p1`; otherwise, the behavior is undefined.
void* narrow_reachable_range_to(void* p1,
                                const void* p2,
                                const void* p3);
```

The same library function could also be available in C; for example, it could be in the `<stdlib.h>` header. The previously given example would then become:

```
int f3(void) {
    struct S s;
    s.x = 1;
    f4((int*)narrow_reachable_range_to(&s.y, &s.y, &s.y + 1));
    return s.x * s.x;
}
```

The C++ standard library could provide more convenient (presumably templated) facilities built on top of `narrow_reachable_range_to`.

This paper does not propose to add reachable ranges to the C++ standard, nor a library function similar to `narrow_reachable_range_to`. This Appendix merely aims to describe one possibility as to how the optimizations that the paper seeks to invalidate could be recovered by a future opt-in mechanism.

§ 13 References

- [CHERI] Robert N. M. Watson et al. 2020-06. CHERI C/C++ Programming Guide.
<https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-947.pdf>
- [CWG2901] Jan Schultke. 2024-06-14. Unclear semantics for near-match aliased access.
<https://wg21.link/cwg2901>
- [N3057] Jens Gustedt et al. 2022-09-20. Programming languages - A Provenance-aware memory object model for C.
<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3057.pdf>
- [N5001] Thomas Köppe. 2024-12-17. Working Draft, Programming Languages — C++.
<https://wg21.link/n5001>
- [P0137R1] Richard Smith. 2016-06-23. Core Issue 1776: Replacement of class objects containing reference members.
<https://wg21.link/p0137r1>
- [P1839R1] Krystian Stasiowski. 2019-10-02. Accessing Object Representations.
<https://wg21.link/p1839r1>
- [P1839R6] Brian Bi, Krystian Stasiowski, Timur Doumler. 2024-10-14. Accessing object representations.
<https://wg21.link/p1839r6>
- [P1839R7] Timur Doumler, Krystian Stasiowski, Brian Bi. 2025-01. Accessing object representations.
<https://isocpp.org/files/papers/P1839R7.html>
- [P1945R0] Krystian Stasiowski. 2019-10-28. Making More Objects Contiguous.
<https://wg21.link/p1945r0>
- [P2795R5] Thomas Köppe. 2024-03-22. Erroneous behaviour for uninitialized reads.
<https://wg21.link/p2795r5>
- [P2883R0] Alisdair Meredith. 2023-05-19. `offsetof` Should Be A Keyword In C++26.
<https://wg21.link/p2883r0>
- [P3425R0] Lewis Baker. 2024-10-16. Reducing operation-state sizes for subobject child operations.
<https://wg21.link/p3425r0>

-
1. In cases where the macro's name is precisely `container_of`, it appears that it usually refers to the version defined by the Linux kernel. This version uses `void*`, not `char*`; pointer arithmetic using `void*` is not proposed by this paper. However, `char*` is used in many other cases.↵

2. All citations to the Standard are to working draft N5001 unless otherwise specified.↵
3. In C, `void` is an object type.↵
4. Note that the Provenance TS does not state that two different complete objects always have different storage IDs. According to section 3.20, a single allocation creates a single storage instance. For example, when `malloc` succeeds, it returns a pointer to “the allocated storage instance” (per section 7.22.3.4).↵
5. An example of dangerous UB is reading from uninitialized variables. I’ve observed recent versions of Clang eliding branches along which uninitialized variables are read, causing unit tests to fail when Clang was upgraded. Such behavior will become (mostly) disallowed in C++26 due to the adoption of [P2795R5].↵
6. §6.2.6.1p7 in [N3057] refers to the “byte array of the storage instance”, implying that pointer arithmetic can be used to traverse the entire storage instance. However, a pointer to the first element of `buf` does not appear to be specified to be interchangeable with a pointer to the corresponding element of the byte array of the storage instance. The latter value must be obtained from the former through conversion, as in Case C. In Case B, §6.5.4p6 of the C23 draft would appear to apply; it states that “A cast that specifies no conversion has no effect on the type or value of the expression”. Therefore, casting from `char*` to `char*` behaves as if the cast were absent.↵